

```

>>> purse > tipJar # Compare WizCoin objects with a comparison operator.
True
>>> purse < tipJar
False
>>> purse > 1000 # Compare with an int.
True
>>> purse <= 1000
False
>>> purse == 1141
True
>>> purse == 1141.0 # Compare with a float.
True
>>> purse == '1141' # The WizCoin is not equal to any string value.
False
>>> bagOfKnuts = wizcoin.WizCoin(0, 0, 1141)
>>> purse == bagOfKnuts
True
>>> purse == (2, 5, 10) # We can compare with a 3-integer tuple.
True
>>> purse >= [2, 5, 10] # We can compare with a 3-integer list.
True
>>> purse >= ['cat', 'dog'] # This should cause an error.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Al\Desktop\wizcoin.py", line 265, in __ge__
    return self._comparisonOperatorHelper(operator.ge, other)
  File "C:\Users\Al\Desktop\wizcoin.py", line 237, in _comparisonOperatorHelper
    otherValue = (other[0] * 17 * 29) + (other[1] * 29) + other[2]
IndexError: list index out of range

```

Our helper method calls `isinstance(other, collections.abc.Sequence)` to see whether `other` is a sequence data type, such as a tuple or list. By making `WizCoin` objects comparable with sequences, we can write code such as `purse >= [2, 5, 10]` for a quick comparison.